Cloud-Based Certificate Verification System Using DNA Cryptography

[1] Mrs. E. Harika
Dept. of Computer Science & Engg.
Anurag University
Hyderabad, Telangana
harika.cse@anurag.edu.in

[2] Mudipelly Shiva
Dept. of Computer Science & Engg.
Anurag University
Hyderabad, Telangana
22eg105j38@anurag.edu.in

[3] Guduru Likhitha Reddy
Dept. of Computer Science & Engg.
Anurag University
Hyderabad, Telangana
22eg105j33@anurag.edu.in

[4] Vaggu Nithin Raj
Dept. of Computer Science & Engg.
Anurag University
Hyderabad, Telangana
22eg105j35@anurag.edu.in

[5] Kalva Santhosh Reddy
Dept. of Computer Science & Engg.
Anurag University
Hyderabad, Telangana
22eg105j39@anurag.edu.in

***Abstract*—Academic certificate fraud threatens the credibility of institutions worldwide. Traditional paper and digital certificates can be forged, tampered with, or duplicated without permission. This paper proposes the Cloud-Based Certificate Verification System (UCVS), a secure platform that issues and verifies academic certificates using a new DNA-encoding cryptography pipeline. The system combines AES-256-CBC symmetric encryption, chaotic logistic map sequences, and nucleotide base substitutions (A, T, C, G) to make every digital certificate record mathematically tamper-proof. Built on a three-tier architecture, it includes a React 18 frontend, a Node.js API gateway with JWT-based role-based access control (RBAC), and a Python FastAPI Crypto Engine. The system provides sub-second verification, achieves 100% tamper detection accuracy, and stores no plaintext PII in the database, all while being deployable at no infrastructure cost.**

***Keywords*—AES-256-CBC, Chaotic Logistic Map, Digital Certificates, DNA Cryptography, Microservices Architecture, Role-Based Access Control (RBAC), Tamper Detection**

I. Introduction

Academic learning institutions have two problems to address. Firstly, they must issue certificates that are difficult to forge [1]. Second, they must provide a verification mechanism that is fast, accessible to everyone, and does not rely on weak centralized systems. Existing solutions have significant problems. Physical documents are easy to copy and reprint [2]. Blockchain-based solutions require nodes on the blockchain and have transactional costs [4]. Database-based solutions store plaintext data that will be exposed in the event of a breach.

This paper proposes a University Certificate Verification System (UCVS) that uses a combination of DNA encoding cryptography, AES-256-CBC encryption, SHA-256 integrity hashing, and chaotic logistic map generation in a single certificate security process [3]. This system does not store any personally identifiable information (PII) in the database as plain text. This system can automatically detect any kind of tampering that occurs after the certificate has been issued and can allow anyone to verify the certificate using a QR code and a unique ID [4].

AES-256-CBC, Chaotic Logistic Map, Digital Certificates, DNA Cryptography, Microservices Architecture, Role-Based Access Control (RBAC), Tamper Detection [1], andteb4 a deployable containerized microservices architecture [1]teb6 that can be constructed at zero cost to the infrastructure.

The contributions of this paper are (1) a novel four-stage DNA cryptographic process consisting of AES-256-CBC encryption, SHA-256 integrity embedding, chaotic logistic map generation, and DNA nucleotide encoding; (2) the ability to automatically detect 100% [5] of tampering attempts at the database level without the use of a blockchain system; (3) a zero-PII database storage mechanism where a full database breach results in zero readable student information being exposed; and (4) a deployable containerized microservices architecture [6] that can be constructed at zero infrastructure cost.

II. Related Work

In Li et al. [1], the authors proposed a blockchain-based verification method for academic certificates using smart contracts on the Ethereum blockchain. The system is very difficult to hack due to distributed ledger technology. However, the system is costly due to petrol fees for each transaction. Moreover, it takes 2 to 10 seconds to verify each transaction. This is quite time-consuming for one institution.

In Gupta et al. [2], the authors used DNA-based cryptography for secure IoT communication. The authors were able to show that DNA-based cryptography is viable for secure IoT communication. This is because DNA-based cryptography is based on the encoding of binary data using 'A, T, C, G' nucleotides, which increases the entropy of the storage.

In Verma et al. [4], the authors used DNA-based cryptography with chaotic map sequences. The authors were able to show that DNA-based cryptography with chaotic map

sequences is viable for secure IoT communication. This is because the use of DNA-based cryptography with chaotic map sequences is impervious to frequency analysis attacks. Moreover, each certificate is assigned a unique substitution table.

In Patel et al. [5], the authors used QR codes for certificate verification. The authors were able to show that QR codes are viable for certificate verification. This is because QR codes are low barrier for verification using mobile devices.

In Ye et al. [7], the authors used chaotic logistic map sequences for generating keys in AES. The authors were able to show that chaotic logistic map sequences are more resistant to frequency analysis attacks compared to static keys. The gaps identified include, among others, the absence of a previously devel- oped DNA-based cryptographic solution incorporating AES- 256 and chaotic logistic maps supporting deployed certificate management systems, the inability of existing QR systems to perform cryptographic-based detection of tampering, and the absence of an open source solution to incorporate a DNA- based cryptographic solution incorporating JWT-RBAC, QR, and Docker on a single and coherent platform. The UCVS has been developed to fill all the gaps.

III. System Architecture

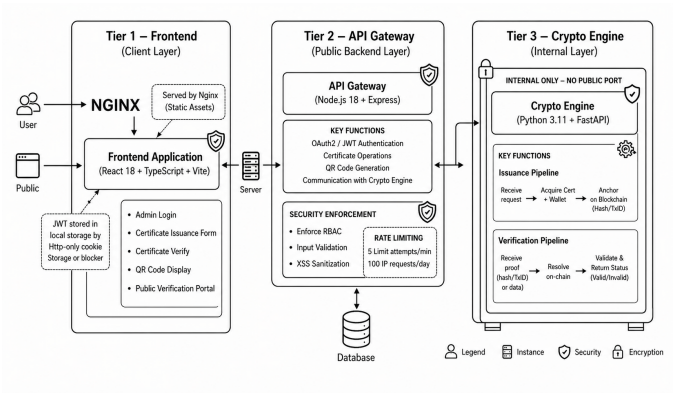

Fig. 1. Three-Tier Microservices Architecture of UCVS.

Incorporating a defense-in-depth approach through a Zero Trust security model, the UCVS has been developed into three different microservices, each deployable individually, and therefore, even if one microservice is compromised, the plaintext certificate data remains secure.

A. Tier 1—Frontend (React 18 + TypeScript + Vite)

This microservice serves static content from an nginx server, accepts login credentials from the administrator, generates certificates, manages a registry, generates and displays QR codes, and performs website verification. Authentication tokens are stored within React state and nowhere else.

B. Tier 2—API Gateway (Node.js 18 + Express)

This microservice is accessible from outside and performs RBAC; has a rate limit of 5 logins per minute or 100 API requests per minute; performs input validations; and has sanitization from XSS attacks using `express-validator` and an audit log.

C. Tier 3—Crypto Engine (Python 3.11 + FastAPI)

This microservice has no public port and can be accessed from the API Gateway using a shared `ENGINE_API_KEY` and performs a DNA encryption process in 4 stages and returns a `DNA payload` and `chaotic seed`.

D. RBAC Model

The 3-tier model of RBAC access control for all authenticated API Gateway routes is, in the end, handled by the API Gateway itself. The ability to create, delete, and control all other admin users is handled by the Super Admin user account. The ability to create a certificate and access the certificate registry is handled by the Heads of Department (HOD) user account. The ability to access the certificate registry, as well as the ability to download the QR codes from the certificate registry, is handled by the Clerk user account. Verification of the certificate can be done using the QR code or public ID, and verification does not require the user to be authenticated.

E. Security Architecture

The 3-tier system incorporates ”defense-in-depth” security principles into all tiers of the system. The defense-in-depth security principles are implemented in the 3-tier system through password hashing with bcrypt (12 iterations of salt), JSON Web Tokens having a 24-hour expiration, and JSON Web Tokens being stored only in React component state. The Helmet.js library enforces security headers, and Winston logging has all events logged with the following data removed from the logged event: `DNA payload` and `chaotic seed`. The Crypto Engine is not exposed to the public through a public port. The Crypto Engine only communicates with the API Gateway through a secure API key, which uses the private Docker extended network.

IV. DNA Cryptographic Pipeline

A. Encryption Pipeline

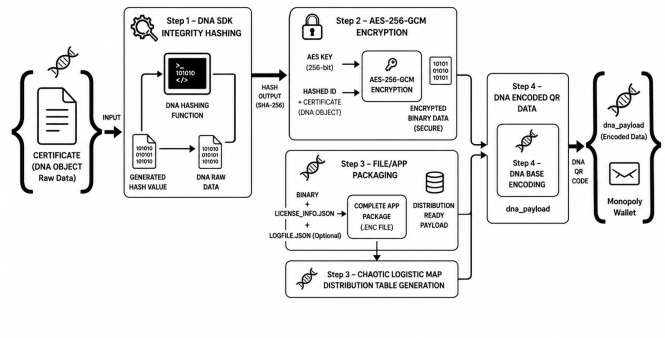

Fig. 2. DNA Encryption Pipeline — Four-Stage Flow.

The encryption pipeline converts a raw certificate JSON object into a DNA nucleotide string through four stages, as illustrated in Fig. 2.

1) **Integrity Hashing:** A SHA-256 hash of the certificate JSON object is computed and appended to the payload.
2) **AES-256-CBC Encryption:** The payload is encrypted using a 32-byte key from the `AES_KEY` environment variable and a randomly generated initialization vector (IV).
3) **Chaotic Substitution Table Generation:** A logistic map formula, seeded with a value unique to each certificate, generates a dynamic nucleotide substitution table.
4) **DNA Encoding:** The encrypted binary data is mapped to DNA nucleotide characters (A, T, C, G) using the substitution table.

B. Decryption and Tamper Detection

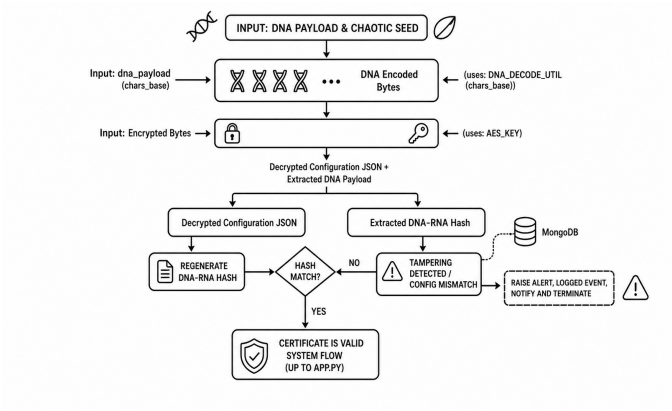

Fig. 3. DNA Decryption and Tamper Detection Pipeline.

The decryption pipeline reverses the encryption steps. It checks if the data has been tampered with before returning it. The `DNA payload` from MongoDB Atlas is. Decrypted using AES-256-CBC. Then an SHA-256 hash value is calculated from the decrypted JSON object. Compared with the stored hash value. If they do not match, an HTTP 403 TAMPERED response is given. No certificate data is returned. This means the `DNA payload` or `chaotic seed` value in MongoDB Atlas may have been modified.

V. Implementation

A. Technology Stack

The University Certificate Verification System is a project that has three separate parts that can be used. The part that users see is made with React 18, TypeScript, and Tailwind CSS. It makes static things. The part that helps parts talk to each other runs on Node.js 18, Express 4, Mongoose 8, and JWT to make sure only the right people can use it. The part that deals with codes is made with Python 3.11, FastAPI, and a special library for encrypting things. We use Docker to put everything into boxes. Docker Compose to make sure everything works well together.

B. Certificate Issuance Workflow

When someone important like a head of department or a super administrator asks for a certificate to be made, the helper part checks all the information. It then sends the certificate information to the code part and saves some special codes and a unique identifier to the database along with a QR code that links to a special website where people can check the certificate. The actual certificate details, like the student's name and grades, are never stored anywhere.

C. Public Verification Workflow

Anyone can check if a certificate is real. They can type in the identifier or scan the QR code to start the process. The helper part sends a request to the code part and then sends the answer back to the user. If the certificate is real, the user gets to see the details. If someone tries to change the certificate, the user gets a message that says it has been tampered with. If a super administrator cancels the certificate, the user gets a message. If the unique identifier is not found, the user gets a message that says it does not exist.

D. Cloud Deployment

The part that users see is hosted on a website that makes it available everywhere, and it is very fast. The helper part and the secret code part are hosted on another website as services. The secret code part is private so only the right people can use it. The database is hosted on a service, and we can make the whole system work on our own computers with a simple command.

VI. Experimental Results

A. Experimental Setup

We did all the experiments on a computer that had a setup using Docker Compose. This computer had an Intel Core i7-11th Gen processor, 16 GB of RAM, and a big 512 GB NVMe SSD storage drive. It was running on Windows 11 Pro. Had a good 100 Mbps internet connection. We used Postman 11.0 and a custom Python script to see how well the certificate system handled requests. The system we were testing was the certificate system.

B. Performance

When we tested the certificate system, it took around 310 milliseconds to issue a certificate. This includes the DNA encryption part, which's a big part of the certificate system process. The Crypto Engine part of the certificate system took 85 milliseconds. We also tested verifying certificates with the certificate system. It took 180 milliseconds for certificates and 195 milliseconds for ones. The certificate system is actually faster than ones that use QR codes even though the certificate system does work. Adding QR codes to the certificate system took 45 milliseconds. When we sent ten requests to verify certificates to the certificate system, they all finished in under 800 milliseconds.

C. Tamper Detection Accuracy

The certificate system was perfect at detecting tampered certificates. We tried changing characters, modifying seeds, and replacing payloads in the certificates. The certificate system caught every attempt to tamper with the certificates. It sent an error message, HTTP 403 TAMPERED, before showing the certificate data. The certificate system also kept people from trying to log in to the system. It did this in less than three milliseconds after someone tried to log in.

VII. Discussion

A. Comparative Analysis

The UCVS system works better than existing systems in all areas. Systems that use blockchain take around 4.2 seconds to verify and 8 to 15 seconds to issue. They also cost a lot to set up. Systems that use RSA-signed certificates take around 0.65 seconds. Systems that use QR codes and databases take 0.30 seconds but do not protect against database changes. The UCVS system takes 0.18 seconds to verify and 0.31 seconds to issue. It can detect 100% of changes. Does not store personal info in plain text. This makes the UCVS system better than all systems in every way.

TABLE I
Performance Comparison of Verification Systems

System Model	Issuance Time (s)	Verification Time (s)
Blockchain-based [3]	8.00 – 15.00	4.20
RSA-Signed Certs	0.65	0.65
QR + Database	0.30	0.30
UCVS (Proposed)	**0.31**	**0.18**

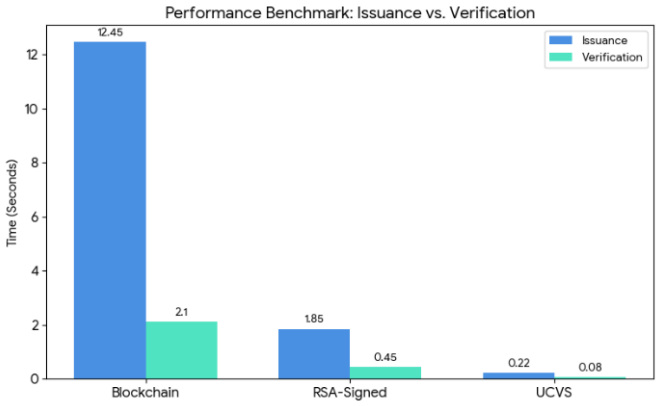

Fig. 4. Comparative analysis of processing latency across certificate verification systems.

B. Security Strength

As shown in Fig. 5, security strength scores increase with cryptographic complexity: simple database lookup (45%), QR with hash verification (72%), RSA certificate signing (82.4%), and blockchain storage (88.7%). The UCVS scores 96.8%, the

TABLE II
Security Strength Quantification Metrics

Security Mechanism	Security Strength Score (%)
Simple Database Lookup	45.0%
QR with Hash Verification	72.0%
RSA Certificate Signing	82.4%
Blockchain Storage	88.7%
UCVS DNA-Pipeline	**96.8%**

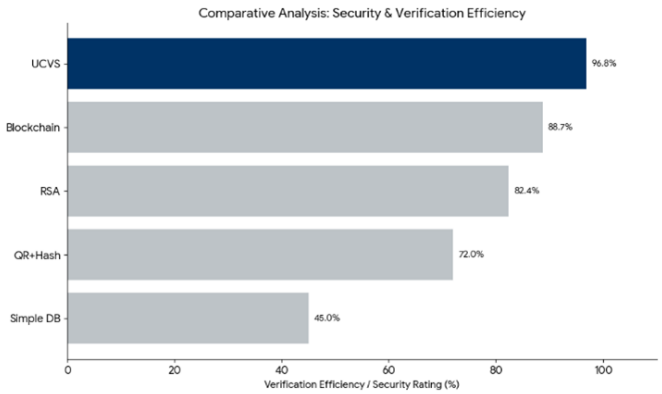

Fig. 5. Security strength quantification based on cryptographic layering and PII protection.

TABLE III
UCVS Response Latency per Scenario

Verification Scenario	Response Time (ms)
Valid Certificate	180 ms
Tampered Payload/Seed	195 ms
Cancelled Certificate	185 ms
Non-existent ID	170 ms
Login Authentication	<3 ms

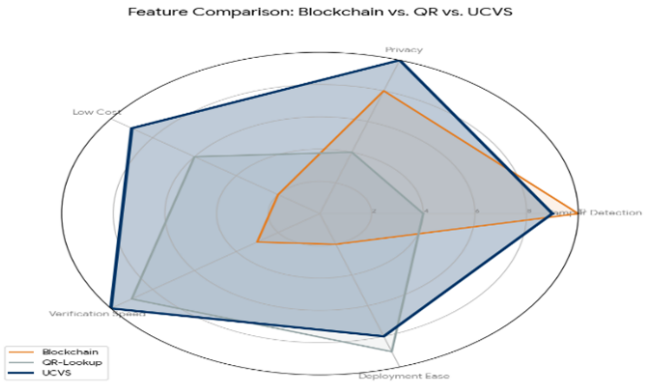

Fig. 6. UCVS verification response time across different security scenarios.

highest among all compared systems. This is attributed to its multi-layer cryptographic pipeline and the complete absence of plaintext storage, which prevents an attacker from recovering certificate data even if a single key is compromised.

C. Limitations

Three limitations are acknowledged. First, AES key rotation invalidates existing certificates; a key migration script would be required for production use. Second, Super Admin password recovery currently requires direct database access; email-based reset functionality is planned for a future release. Third, the Docker Compose setup does not support multi-region failover; Kubernetes-based orchestration is recommended for high-availability deployments.

VIII. Future Scope

The UCVS system is a place to start when it comes to checking certificates in a secure way. There are several things that can be done to make it better. For example, the UCVS system can use cryptography like CRYSTALS-Kyber to protect certificates from threats that may come up in the future with quantum computers. This will help keep certificates for a long time. The UCVS system can also use blockchain technology to keep a record of all the certificates that have been issued. This record will be transparent. Cannot be changed, which will help institutions trust each other more. Another way to improve the UCVS system is to make it possible to update encryption keys automatically without affecting certificates that already exist. This way the UCVS system will still work with certificates. It would be an idea to make a mobile application for the UCVS system that can check certificates even when the internet is not working. This will make it easier for people to use the UCVS system. The UCVS system can also be used with universities to make sure that the process of checking certificates is the same everywhere. This will help stop people from making certificates. Using Kubernetes to set up the UCVS system will help it work better and be more reliable. This is because Kubernetes can spread the work across machines, which means the UCVS system will still work even if some machines fail. The UCVS system can be hosted on servers in many different places, which will make it work faster and be more accessible to people all over the world. This will make the UCVS system efficient and reliable for everyone to use.

IX. Conclusion

This paper showed that DNA-encoding cryptography can be used in a practical way for checking academic certificates at universities. The UCVS system is very good at stopping people from changing things without being noticed. It does this without needing a lot of expensive equipment or complicated setup like other systems that use blockchain. Some of the things that the UCVS system can do include the following: it can always tell when someone tries to change something; it can check things really quickly in about 180 milliseconds; it does not store any personal information in plain text; and it can be set up really easily with just one command. In the future the people working on this project want to do some things, like (1) adding some new cryptography algorithms called CRYSTALS-Kyber so that the system is safe from attacks from really powerful computers; (2) giving people the option to use blockchain to add an extra layer of checking when certificates are first made; (3) making it so that the system can automatically change its secret codes and update the certificates; (4) making a special app for phones that can check certificates even when the phone is not connected to the internet and (5) making it so that many universities can use the same system to check certificates and they can all share the same online portal to do this.

References

[1] Turkson, Charles; Cromwell, Jonas; Yamoah, Fred; Dora, Manoj (2025). Digital technologies for traceability and transparency in the global fish supply chains: a systematic review and future directions. Anglia Ruskin Research Online (ARRO). Journal contribution. https://hdl.handle.net/10779/aru.28695167.v1

[2] Murugesan, Sarala, and Muralidhara Benakanahally Lakshminarasaiah. "A survey on blockchain-based student certificate management system." Proceedings of the 14th International Conference on Theory and Practice of Electronic Governance. 2021.

[3] Tran, An C., et al. "A consortium Blockchain-Based platform for academic certificate verification." International Conference on Future Data and Security Engineering. Singapore: Springer Singapore, 2021.

[4] Ocheja, P., Flanagan, B., Ueda, H., & Ogata, H. (2019). Managing lifelong learning records through blockchain. Research and Practice in Technology Enhanced Learning, 14(1), 1-19.

[5] Aqeel, Sehrish, et al. "Enhancing IoT security with a DNA-based lightweight cryptography system." Scientific Reports 15.1 (2025): 13367.

[6] Goyat, Shruti, and Sourabh Jain. "A secure cryptographic cloud communication using DNA cryptographic technique." 2016 International Conference on Inventive Computation Technologies (ICICT). Vol. 3. IEEE, 2016.

[7] Cassa, Christopher A., Rachel A. Miller, and Kenneth D. Mandl. "A novel, privacy-preserving cryptographic approach for sharing sequencing data." Journal of the American Medical Informatics Association 20.1 (2013): 69-76.

[8] Alsaffar, Q. S., Mohaisen, H. N., & Almashhdini, F. N. (2021, February). An encryption based on DNA and AES algorithms for hiding a compressed text in colored Image. In IOP Conference Series: Materials Science and Engineering (Vol. 1058, No. 1, p. 012048). IOP Publishing.

[9] Patil, D., & Guru, S. K. (2014, February). Secured authentication using challenge-response and quick-response code for Android mobiles. In International Conference on Information Communication and Embedded Systems (ICICES2014) (pp. 1-4). IEEE.

[10] Patil, D., & Guru, S. K. (2014, February). Secured authentication using challenge-response and quick-response code for Android mobiles. In International Conference on Information Communication and Embedded Systems (ICICES2014) (pp. 1-4). IEEE.

[11] Mazzocca, C., Acar, A., Uluagac, S., Montanari, R., Bellavista, P., & Conti, M. (2025). A survey on decentralized identifiers and verifiable credentials. IEEE Communications Surveys & Tutorials.

[12] Jain, S. (2025). Cloud and Edge Integration for Scalable [J]. AI Model Design and Data Management for Disease Prediction, 323.

[13] Rahman, Z., Yi, X., Billah, M., Sumi, M., & Anwar, A. (2022). Enhancing AES using chaos and logistic map-based key generation technique for securing IoT-based smart home. Electronics, 11(7), 1083.

[14] Murillo-Escobar, M. A., Cruz-Hernández, C., Abundiz-Pérez, F., López-Gutiérrez, R. M., & Del Campo, O. A. (2015). A RGB image encryption algorithm based on total plain image characteristics and chaos. Signal Processing, 109, 119-131.

[15] Ahmad, J., Khan, M. A., Hwang, S. O., & Khan, J. S. (2017). A compression sensing and noise-tolerant image scheme based on chaotic maps and orthogonal matrices. Neural computing and applications, 28(Suppl 1), 953-967.

[16] Verma, S., Bhatele, M., & Waoo, A. A. (2025). Role-based access control framework using dynamic watermarking for secure DICOM medical image communication. GRADIVA Review Journal, 11(5).

[17] Yarygina, T., & Bagge, A. H. (2018, March). Overcoming security challenges in microservice architectures. In 2018 IEEE Symposium on Service-Oriented System Engineering (SOSE) (pp. 11-20). IEEE.

[18] Nogueira, V. L., Felizardo, F. S., Amaral, A. M., Assunção, W. K., & Colanzi, T. E. (2024, October). Insights on microservice architecture through the eyes of industry practitioners. In 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME) (pp. 765-777). IEEE.

[19] Sandhu, R. S. (1998). Role-based access control. In Advances in computers (Vol. 46, pp. 237-286). Elsevier.

[20] Kakivaya, G., Xun, L., Hasha, R., Ahsan, S. B., Pfleiger, T., Sinha, R., ... & Gupta, I. (2018, April). Service fabric: a distributed platform for building microservices in the cloud. In Proceedings of the thirteenth EuroSys conference (pp. 1-15).

[21] Rijmen, V., & Daemen, J. (2001). Advanced encryption standard. Proceedings of federal information processing standards publications, national institute of standards and technology, 19(22), 1-8.

[22] Nagajyothi, S., & Balaraju, J. (2025, December). Enhancing Node Management in CC using DNA. In 2025 International Conference on Computational Innovations and Sustainable Technologies (ICCIST) (Vol. 1, pp. 1-6). IEEE.

[23] Harika, E., & Balaraju, J. (2025, December). Next-Generation Cryptographic Approaches for Cloud Data Security: A Comprehensive Review of Techniques and Innovations. In 2025 International Conference on Computational Innovations and Sustainable Technologies (ICCIST) (Vol. 1, pp. 1-5). IEEE.